

Core Language Working Group NB comment resolutions for the November, 2022 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4917](#).

2242. ODR violation with constant initialization possibly omitted

Section: 6.3 [[basic.def.odr](#)] **Status:** NB **Submitter:** Hubert Tong **Date:** 2016-03-05 [P2720R0 comment US 5-033](#)

Consider the following example:

```
// tu1.cpp
extern const int a = 1;
inline auto f() {
    static const int b = a;
    struct A { auto operator()() { return &b; } } a;
    return a;
}

// tu2.cpp
extern const int a;
inline auto f() {
    static const int b = a;
    struct A { auto operator()() { return &b; } } a;
    return a;
}
int main() {
    return *decltype(f())();
}
```

Here, b may or may not have constant initialization. This example should be an ODR violation.

(Split off from issue 2123.)

Proposed resolution (approved by CWG 2022-11-11):

Insert after 6.3 [[basic.def.odr](#)] bullet 14.7 as follows:

- ...
- In each such definition, corresponding entities shall have the same language linkage.
- **In each such definition, const objects with static or thread storage duration shall be constant-initialized if the object is constant-initialized in any such definition.**
- ...

2538. Can standard attributes be syntactically ignored?

Section: 9.12.1 [[dcl.attr.grammar](#)] **Status:** NB **Submitter:** Jens Maurer **Date:** 2021-12-02 [P2720R0 comment GB 055](#)

Subclause 9.12.1 [[dcl.attr.grammar](#)] paragraph 6 specifies that an unrecognized *attribute-token* is ignored:

For an *attribute-token* (including an *attribute-scoped-token*) not specified in this document, the behavior is implementation-defined. Any *attribute-token* that is not recognized by the implementation is ignored.

The intent is that only non-standard unrecognized *attribute-tokens* can be ignored; in particular, an implementation is required to syntax-check all standard attributes, even if the implementation then chooses not to effect any semantics for that attribute.

The paper introducing attributes was [N2761](#); the phrasing in question was introduced by [P0283R2](#) attempting to implement the design change presented in [P0283R1](#).

See also paper [P2552](#) (On the ignorability of standard attributes).

Suggested resolution:

Change in 9.12.1 [[dcl.attr.grammar](#)] paragraph 6 as follows:

For an *attribute-token* (including an *attribute-scoped-token*) not specified in this document, the behavior is implementation-defined: ~~Any~~ ; **any such** *attribute-token* that is not recognized by the implementation is ignored.

EWG telecon 2022-05-26

See [paper issue 1252](#).

There was consensus for the statement "It is EWG's intent that [dcl.attr]/6 ONLY permits an implementation to ignore a standard attribute's effect, but not appertainment and argument parsing." To be confirmed by electronic polling.

Proposed resolution (approved by CWG 2022-07-01) [SUPERSEDED]:

Change in 9.12.1 [[dcl.attr.grammar](#)] paragraph 6 as follows:

For an *attribute-token* (including an *attribute-scoped-token*) not specified in this document, the behavior is implementation-defined: ~~Any~~ ; **any such** *attribute-token* that is not recognized by the implementation is ignored. **[Note: A program is ill-formed if it contains an *attribute* specified in 9.12 [[dcl.attr](#)] that violates the rules to which entity or statement the *attribute* may apply or the syntax rules for the *attribute's attribute-argument-clause*, if any. -- end note]**

EWG 2022-06 electronic polling

No consensus. See [vote](#).

EWG 2022-11-08

Approved the direction of the 2022-07-01 proposed resolution.

Proposed resolution (approved by CWG 2022-11-08):

Change in 9.12.1 [[dcl.attr.grammar](#)] paragraph 6 as follows:

For an *attribute-token* (including an *attribute-scoped-token*) not specified in this document, the behavior is implementation-defined: ~~Any~~ ; **any such** *attribute-token* that is not recognized by the implementation is ignored. **[Note: A program is ill-formed if it contains an *attribute* specified in 9.12 [[dcl.attr](#)] that violates the rules specifying to which entity or statement the *attribute* may apply or the syntax rules for the *attribute's attribute-argument-clause*, if any. -- end note]**

2615. Missing `__has_cpp_attribute(assume)`

Section: 15.2 [[cpp.cond](#)] **Status:** NB **Submitter:** S. B. Tam **Date:** 2022-08-17 [P2720R0 comment GB 070](#)

Paper P1774R8 (accepted in July, 2022) adds a new attribute `assume`, but neglects to update table 22 in 15.2 [[cpp.cond](#)].

Proposed resolution (accepted by CWG 2022-08-26):

In 15.2 [[cpp.cond](#)], add a row to table `tab:cpp.cond.ha` as follows:

Attribute	Value
<code>assume</code>	<code>202207L</code>

2621. Kind of lookup for using enum declarations

Section: 9.7.2 [[enum.udecl](#)] **Status:** NB **Submitter:** Sean Baxter **Date:** 2022-08-17 [P2720R0 comment CA 054](#)

Consider:

```
enum class E {  
    a, b, c  
};  
  
using MyE = E;  
  
int main() {  
    using enum MyE;    // #1  
}
```

Does the lookup for the *elaborated-enum-specifier* at #1 use type-only lookup per 6.5.6 [[basic.lookup.elab](#)]? There is implementation divergence; EDG, gcc, and MSVC accept, clang rejects.

Suggested resolution [SUPERSEDED]:

Change in 9.7.2 [[enum.udecl](#)] paragraph 1 as follows:

Lookup for the *elaborated-enum-specifier* is as specified in 6.5.6 [[basic.lookup.elab](#)]. The *elaborated-enum-specifier* shall not name a dependent type and the type shall have a reachable *enum-specifier*.

CWG telecon 2022-09-09:

The example at #1 is intended to be valid; even though the grammar suggests that an *elaborated-type-specifier* is used here, an ordinary (not a type-only) lookup is performed.

Proposed resolution (approved by CWG 2022-09-23) [SUPERSEDED]:

Change in 9.7.2 [[enum.udecl](#)] paragraph 1 as follows:

The terminal name of the *elaborated-enum-specifier* undergoes ordinary lookup (6.5.3 [[basic.lookup.unqual](#)], 6.5.5 [[basic.lookup.qual](#)]). The *elaborated-enum-specifier* shall not name a dependent type and the type shall have a reachable *enum-specifier*.

CWG 2022-11-07:

Use the wording approach presented in CA-054, with necessary adjunct fixes.

Proposed resolution (approved by CWG 2022-11-10):

Change the grammar before 9.2.9.4 [[dcl.type.elab](#)] paragraph 1 as follows, merging the grammar productions:

```
elaborated-type-specifier :  
    class-key attribute-specifier-seqopt nested-name-specifieropt identifier  
    class-key simple-template-id  
    class-key nested-name-specifier templateopt simple-template-id  
elaborated enum specifier  
elaborated enum specifier :  
    enum nested-name-specifieropt identifier
```

Change the grammar before 9.7.2 [[enum.udecl](#)] paragraph 1 as follows:

```
using-enum-declaration :  
    using elaborated enum specifier enum using-enum-declarator ;  
  
using-enum-declarator :  
    nested-name-specifieropt identifier  
    nested-name-specifieropt simple-template-id
```

Change in 9.7.2 [[enum.udecl](#)] paragraph 1 as follows:

A *using-enum-declarator* names the set of declarations found by lookup (6.5.3 [basic.lookup.unqual], 6.5.5 [basic.lookup.qual]) for the *using-enum-declarator*. The *elaborated-enum-specifier* *using-enum-declarator* shall not name a dependent type and the type shall have with a reachable *enum-specifier*.

2631. Immediate function evaluations in default arguments

Section: 7.7 [expr.const] **Status:** NB **Submitter:** Aaron Ballman **Date:** 2022-09-16

Consider:

```
constexpr int const_div(int a, int b) { return a / b; }
int func(int x = const_div(10, 0));
```

According to 7.7 [expr.const] paragraph 14:

An expression or conversion is an *immediate invocation* if it is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

Subclause 9.3.4.7 [dcl.fct.default] paragraph 5 specifies:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [dcl.init]). The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears. Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [temp.inst].

Checking the semantic constraints of the default argument appears to include a check whether the immediate invocation of `const_div` is actually a constant expression, even though the default argument's value might never actually be used for any function call in the program.

However, instantiation of a `constexpr` function template to be able to perform the constant evaluation is not permitted per 13.9.2 [temp.inst] paragraph 11:

... The use of a template specialization in a default argument shall not cause the template to be implicitly instantiated except that a class template may be instantiated where its complete type is needed to determine the correctness of the default argument. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated.

Example 2:

```
constexpr int g();
constexpr int f() {
    return g();
}

int k(int x = f()) { // error: constexpr evaluation of undefined function g
    return x;
}

constexpr int g() {
    return 42;
}

int main() {
    return k();
}
```

Example 3:

```
#include <source_location>
#include <iostream>

constexpr int const_div(int a, int b) {
    return a / b;
}

#line 5
void foo(int x = const_div(1000, std::source_location::current().line() - 15)) {
```

```

    std::cout << x << "\n";
}

// Should the definition of `bar` produce errors? (division by zero during constant
// evaluation for constraint checking)
#line 10
void bar(int x = const_div(1000, std::source_location::current().line() - 10)) {
    std::cout << x << "\n";
}

int main() {
    // Should this call produce errors? (division by zero during constant
    // evaluation of the default argument)
    #line 15
    foo();
    #line 20
    bar();
}

```

Note that `source_location::current()` is specified to take its value from the location where it is evaluated, if it appears in a default argument (17.8.2.2 [support.srcloc.cons] paragraph 2):

Remarks: Any call to `current` that appears as a default member initializer (11.4 [class.mem]), or as a subexpression thereof, should correspond to the location of the constructor definition or aggregate initialization that uses the default member initializer. Any call to `current` that appears as a default argument (9.3.4.7 [dcl.fct.default]), or as a subexpression thereof, should correspond to the location of the invocation of the function that uses the default argument (7.6.1.3 [expr.call]).

Proposed resolution (September, 2022) [SUPERSEDED]:

1. Split 9.3.4.7 [dcl.fct.default] paragraph 5 and change as follows:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [dcl.init]).

A default argument context is

- the initializer-clause in a parameter-declaration, including any conversions to the parameter type,
- a subexpression of one of the above that is not a subexpression of a nested unevaluated operand.

The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, **except that an immediate invocation (7.7 [expr.const]) in a default argument context is neither evaluated nor checked for whether it is a constant expression at that point**. Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [temp.inst].

2. Change in 13.9.2 [temp.inst] paragraph 11 as follows:

... The use of a template specialization in a default argument shall not cause the template to be implicitly instantiated except that a class template **or a function template with a deduced return type** may be instantiated where its complete type is needed to determine the correctness of the default argument. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated.

Approved by EWG telecon 2022-09-29.

Amendment to also cover default member initializers (October, 2022) [SUPERSEDED]:

1. Split 9.3.4.7 [dcl.fct.default] paragraph 5 and change as follows:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [dcl.init]).

A default argument context of an initializer is

- the initializer, including any conversions to the target type, or
- a subexpression thereof that is not a subexpression of a nested unevaluated operand.

The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, **except that an immediate invocation (7.7 [expr.const]) in a default argument context of the initializer-clause in a parameter-declaration is neither evaluated nor checked for whether it is a constant expression at that point**. Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [temp.inst].

2. Change in 11.4.1 [[class.mem.general](#)] paragraph 11 as follows:

A *brace-or-equal-initializer* for a non-static data member specifies a *default member initializer* for the member, and shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor. **An immediate invocation (7.7 [[expr.const](#)]) in a default argument context (9.3.4.7 [[dcl.fct.default](#)]) of a default member initializer is neither evaluated nor checked for whether it is a constant expression at the point where it appears.**

3. Change in 13.9.2 [[temp.inst](#)] paragraph 11 as follows:

... The use of a template specialization in a default argument shall not cause the template to be implicitly instantiated except that a class template **or a function template with a deduced return type** may be instantiated where its complete type is needed to determine the correctness of the default argument. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated.

CWG telecon 2022-10-07:

The new term should be defined in 6.9.1 [[intro.execution](#)] without reference to immediacy.

Possible resolution [[SUPERSEDED](#)]:

1. Change in 6.9.1 [[intro.execution](#)] paragraph 4 as follows:

A *subexpression* of an expression E is an immediate subexpression of E or a subexpression of an immediate subexpression of E. [Note: ... -- end note]

The *potentially-evaluated subexpressions* of an expression, conversion, or *initializer* E are

- the constituent expressions of E and
- the subexpressions thereof that are not subexpressions of a nested unevaluated operand (7.2.3 [[expr.context](#)]).

2. Change in 7.7 [[expr.const](#)] paragraph 16 as follows:

An expression or conversion is *potentially constant evaluated* if it is:

- ...
- a **potentially-evaluated** subexpression (6.9.1 [[intro.execution](#)]) of one of the above ~~that is not a subexpression of a nested unevaluated operand (7.2.3 [[expr.context](#)]).~~

3. Change in 9.3.4.7 [[dcl.fct.default](#)] paragraph 5 as follows:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [[dcl.init](#)]). The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, **except that an immediate invocation (7.7 [[expr.const](#)]) that is a potentially-evaluated subexpression (6.9.1 [[intro.execution](#)]) of the initializer-clause in a parameter-declaration is neither evaluated nor checked for whether it is a constant expression at that point.** Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [[temp.inst](#)].

4. Add a paragraph before 9.4.1 [[dcl.init.general](#)] paragraph 17 as follows:

An immediate invocation (7.7 [[expr.const](#)]) that is not evaluated where it appears (9.3.4.7 [[dcl.fct.default](#)], 11.4.1 [[class.mem.general](#)]) is evaluated and checked for whether it is a constant expression at the point where the enclosing *initializer* is used in a function call, a constructor definition, or an aggregate initialization.

An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4 [[temp.variadic](#)]).

5. Change in 11.4.1 [[class.mem.general](#)] paragraph 11 as follows:

A *brace-or-equal-initializer* for a non-static data member specifies a *default member initializer* for the member, and shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor. **An immediate invocation (7.7 [[expr.const](#)]) that is a potentially-evaluated subexpression (6.9.1 [[intro.execution](#)]) of a default member initializer is neither evaluated nor checked for whether it is a constant expression at the point where the subexpression appears.**

6. Change in 13.9.2 [[temp.inst](#)] paragraph 11 as follows:

... The use of a template specialization in a default argument **or default member initializer** shall not cause the template to be implicitly instantiated except that a **templated** class ~~template~~, **a templated function with a deduced**

return type, or a variable template with a deduced type may be instantiated where its complete type is needed to determine the correctness of the default argument **or default member initializer**. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated. **Similarly, the use of a default member initializer in a constructor definition or an aggregate initialization causes specializations in the default member initializer to be instantiated.**

CWG telecon 2022-10-21:

In the above wording, "constituent expression of a conversion" is not a defined term.

Possible resolution [SUPERSEDED]:

1. Change in 6.9.1 [intro.execution] paragraph 2 as follows:

A constituent expression is defined as follows:

- The constituent expression of an expression is that expression.
- **The constituent expression of a conversion is the corresponding implicit function call, if any, or the converted expression otherwise.**
- The constituent expressions of a *braced-init-list* or of a (possibly parenthesized) *expression-list* are the constituent expressions of the elements of the respective list.
- The constituent expressions of a *brace-or-equal-initializer* of the form *= initializer-clause* are the constituent expressions of the *initializer-clause*.

2. Change in 6.9.1 [intro.execution] paragraph 4 as follows:

A *subexpression* of an expression E is an immediate subexpression of E or a subexpression of an immediate subexpression of E. [Note: ... -- end note]

The *potentially-evaluated subexpressions* of an expression, conversion, or *initializer* E are

- the constituent expressions of E and
- the subexpressions thereof that are not subexpressions of a nested unevaluated operand (7.2.3 [expr.context]).

3. Change in 7.7 [expr.const] paragraph 16 as follows:

An expression or conversion is *potentially constant evaluated* if it is:

- ...
- a **potentially-evaluated** subexpression (6.9.1 [intro.execution]) of one of the above ~~that is not a subexpression of a nested unevaluated operand (7.2.3 [expr.context]).~~

4. Change in 9.3.4.7 [dcl.fct.default] paragraph 5 as follows:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [dcl.init]). The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, **except that an immediate invocation (7.7 [expr.const]) that is a potentially-evaluated subexpression (6.9.1 [intro.execution]) of the initializer-clause in a parameter-declaration is neither evaluated nor checked for whether it is a constant expression at that point.** Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [temp.inst].

5. Add a paragraph before 9.4.1 [dcl.init.general] paragraph 17 as follows:

An immediate invocation (7.7 [expr.const]) that is not evaluated where it appears (9.3.4.7 [dcl.fct.default], 11.4.1 [class.mem.general]) is evaluated and checked for whether it is a constant expression at the point where the enclosing *initializer* is used in a function call, a constructor definition, or an aggregate initialization.

An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4 [temp.variadic]).

6. Change in 11.4.1 [class.mem.general] paragraph 11 as follows:

A *brace-or-equal-initializer* for a non-static data member specifies a *default member initializer* for the member, and shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor. **An immediate invocation (7.7 [expr.const]) that is a potentially-evaluated subexpression (6.9.1 [intro.execution]) of a default member initializer is neither evaluated nor checked for whether it is a constant expression at the point where the subexpression appears.**

7. Change in 13.9.2 [temp.inst] paragraph 11 as follows:

... The use of a template specialization in a default argument **or default member initializer** shall not cause the template to be implicitly instantiated except that a **templated class template, a templated function with a deduced return type, or a variable template with a deduced type** may be instantiated where its complete type is needed to determine the correctness of the default argument **or default member initializer**. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated. **Similarly, the use of a default member initializer in a constructor definition or an aggregate initialization causes specializations in the default member initializer to be instantiated.**

CWG 2022-11-07

Expand requirement to avoid template instantiations in default arguments by not giving a specific, closed list.

Proposed resolution (approved by CWG 2022-11-08):

1. Change in 6.9.1 [[intro.execution](#)] paragraph 2 as follows:

A constituent expression is defined as follows:

- The constituent expression of an expression is that expression.
- **The constituent expression of a conversion is the corresponding implicit function call, if any, or the converted expression otherwise.**
- The constituent expressions of a *braced-init-list* or of a (possibly parenthesized) *expression-list* are the constituent expressions of the elements of the respective list.
- The constituent expressions of a *brace-or-equal-initializer* of the form = *initializer-clause* are the constituent expressions of the *initializer-clause*.

2. Change in 6.9.1 [[intro.execution](#)] paragraph 4 as follows:

A *subexpression* of an expression E is an immediate subexpression of E or a subexpression of an immediate subexpression of E. [Note: ... -- end note]

The *potentially-evaluated subexpressions* of an expression, conversion, or *initializer* E are

- the constituent expressions of E and
- the subexpressions thereof that are not subexpressions of a nested unevaluated operand (7.2.3 [[expr.context](#)]).

3. Change in 7.7 [[expr.const](#)] paragraph 16 as follows:

An expression or conversion is *potentially constant evaluated* if it is:

- ...
- a **potentially-evaluated** subexpression (6.9.1 [[intro.execution](#)]) of one of the above ~~that is not a subexpression of a nested unevaluated operand (7.2.3 [[expr.context](#)]).~~

4. Change in 9.3.4.7 [[dcl.fct.default](#)] paragraph 5 as follows:

The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.4 [[dcl.init](#)]). The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, **except that an immediate invocation (7.7 [[expr.const](#)]) that is a potentially-evaluated subexpression (6.9.1 [[intro.execution](#)]) of the initializer-clause in a parameter-declaration is neither evaluated nor checked for whether it is a constant expression at that point.** Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2 [[temp.inst](#)].

5. Add a paragraph before 9.4.1 [[dcl.init.general](#)] paragraph 17 as follows:

An immediate invocation (7.7 [[expr.const](#)]) that is not evaluated where it appears (9.3.4.7 [[dcl.fct.default](#)], 11.4.1 [[class.mem.general](#)]) is evaluated and checked for whether it is a constant expression at the point where the enclosing *initializer* is used in a function call, a constructor definition, or an aggregate initialization.

An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4 [[temp.variadic](#)]).

6. Change in 11.4.1 [[class.mem.general](#)] paragraph 11 as follows:

A *brace-or-equal-initializer* for a non-static data member specifies a *default member initializer* for the member, and shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor. **An immediate invocation (7.7 [[expr.const](#)]) that is a potentially-evaluated subexpression (6.9.1 [[intro.execution](#)]) of a default member initializer is neither evaluated nor checked for whether it is a constant expression at the point where the subexpression appears.**

7. Change in 13.9.2 [[temp.inst](#)] paragraph 11 as follows:

... The use of a template specialization in a default argument **or default member initializer** shall not cause the template to be implicitly instantiated except ~~that a class template may be instantiated~~ where its complete type is needed to determine the correctness of the default argument **or default member initializer**. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated. **Similarly, the use of a default member initializer in a constructor definition or an aggregate initialization causes specializations in the default member initializer to be instantiated.**

2636. Update Annex E based on Unicode 15.0 UAX #31

Section: Clause Annex E [[uaxid](#)] **Status:** NB **Submitter:** Steve Downey **Date:** 2022-10-20 [P2720R0 comment US 64-132](#)

Unicode 15.0 UAX #31 clarified that rule R3 was, in fact, intended to apply to programming languages. WG21's prior understanding was that programming languages are not in scope of that rule. The proposed resolution updates E.4 [[uaxid.pattern](#)] to the revised understanding. See paper P2653R1 (Update Annex E based on Unicode 15.0 UAX 31) for more details.

Proposed resolution (approved by CWG 2022-10-21):

Change in E.4 [[uaxid.pattern](#)] as follows:

UAX #31 describes how **formal** languages ~~that use or interpret patterns of characters, such as regular expressions or number formats, may describe that syntax with Unicode properties~~ **such as computer languages should describe and implement their use of whitespace and syntactically significant characters during the processes of lexing and parsing.**

C++ does not ~~do this as part of the language, deferring to library components for such usage of patterns. This requirement does not apply to C++~~ **claim conformance with this requirement.**

2639. new-lines after phase 1

Section: 5.2 [[lex.phases](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 3-030](#)

Translation phases 2 and 3 assume that lines are terminated by "new-line characters". However, the current specification of phase 1 does not guarantee that to be true. In particular, for a UTF-8 file the verbatim sequence of source file characters forms the input for phase 2, even on systems where the line terminator is a carriage return. The non-UTF-8 specification is also defective in that it speaks of "introducing" new-line characters, even for encodings like Latin-1 where new-lines might already be present and no "introduction" is needed or appropriate.

Proposed resolution [SUPERSEDED]:

Change in 5.2 [[lex.phases](#)] paragraph 1.1 as follows:

... If an input file is determined to be a UTF-8 file, then it shall be a well-formed UTF-8 code unit sequence and it is decoded to produce a sequence of UCS scalar values that constitutes the sequence of elements of the translation character set, **representing each line-termination character or character sequence as a new-line character.**

For any other kind of input file supported by the implementation, characters are mapped, in an implementation-defined manner, to a sequence of translation character set elements (5.3 [[lex.charset](#)]) ~~(introducing new-line characters for~~ **representing** end-of-line indicators **as new-line characters**).

Proposed resolution (approved by CWG 2022-11-08):

Change in 5.2 [[lex.phases](#)] paragraph 1.1 as follows:

... If an input file is determined to be a UTF-8 file, then it shall be a well-formed UTF-8 code unit sequence and it is decoded to produce a sequence of UCS scalar values that constitutes the sequence of elements of the translation character set. **In the resulting sequence, each pair of characters in the input sequence consisting of U+000D CARRIAGE**

RETURN followed by U+000A LINE FEED, as well as each U+000D CARRIAGE RETURN not immediately followed by a U+000A LINE FEED, is replaced by a single new-line character.

For any other kind of input file supported by the implementation, characters are mapped, in an implementation-defined manner, to a sequence of translation character set elements (5.3 [lex.charset]) ~~(introducing new-line characters for~~, **representing** end-of-line indicators **as new-line characters**).

2640. Allow more characters in an n-char sequence

Section: 5.3 [lex.charset] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 1-028](#)

The n-char grammar term is defined to match only the Latin uppercase, Latin digit, hyphen and space characters. This results in `\N{ABC}` matching named-universal-character while `\N{abc}` does not. This leads to programs like the following being unexpectedly well-formed because the `\N{abc}` sequence is lexed as the preprocessing token sequence `, N, {, abc, }`. The expansion of macro `a` then leads to the token sequence being passed as an argument to macro `z` where it is discarded.

```
#define z(x) 0
#define a z(
int x = a\N{abc});
```

Changes to make the above program ill-formed would provide two benefits:

- Implementations could diagnose the `\N{abc}` sequence as an ill-formed named-universal-character regardless of where it appears in a program.
- The `\N{...}` syntax space would be reserved for expansion (e.g., for extensions or future support of UAX44-LM2 loose matching schemes).

Proposed resolution (approved by CWG 2022-11-07):

Change the grammar in 5.3 [lex.charset] paragraph 3 as follows:

```
n-char:
  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
  0 1 2 3 4 5 6 7 8 9
  U+002d hyphen-minus
  U+0020 space
  any member of the translation character set except the U+007D RIGHT CURLY BRACKET or new-line character
```

2642. Inconsistent use of T and C

Section: 6.5.2 [class.member.lookup] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 7-035](#)

T and C are used inconsistently throughout these paragraphs.

Proposed resolution:

1. Change in 6.5.2 [class.member.lookup] paragraph 1 as follows:

A search in a scope X for a name N from a program point P is a single search in X for N from P unless X is the scope of a class or class template **T C**, in which case the following steps define the result of the search.

2. Change in 6.5.2 [class.member.lookup] paragraph 4 as follows:

[Note 2: If **T C** is incomplete, only base classes whose *base-specifier* appears before P are considered. If **T C** is an instantiated class, its base classes are not dependent. —end note]

3. Change in 6.5.2 [class.member.lookup] paragraph 6 as follows:

The result of the search is the declaration set of `S(N, T C)`. If it is an invalid set, the program is ill-formed. If it differs from the result of a search in **T C** for N in a complete-class context (11.4 [class.mem]) of **T C**, the program is ill-formed, no diagnostic required.

4. Change in 6.5.2 [[class.member.lookup](#)] paragraph 7 as follows:

If N is a non-dependent *conversion-function-id*, conversion function templates that are members of ~~T~~ C are considered. For each such template F, the lookup set S(t, ~~T~~ C) is constructed, considering a function template declaration to have the name t only if it corresponds to a declaration of F (6.4.1 [[basic.scope.scope](#)]).

5. Change in 6.5.2 [[class.member.lookup](#)] paragraph 8 as follows:

[Note 4: A static member, a nested type or an enumerator defined in a base class ~~T~~ B can unambiguously be found even if an object has more than one base class subobject of type ~~T~~ B. Two base class subobjects share the non-static member subobjects of their common virtual base classes. —end note]

2643. Completing a pointer to array of unknown bound

Section: 6.8.1 [[basic.types.general](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 10-039](#)

Subclause 6.8.1 [[basic.types.general](#)] paragraph 6 specifies:

... The type of a pointer to array of unknown bound, or of a type defined by a typedef declaration to be an array of unknown bound, cannot be completed.

This is misleading; such a type is already complete.

Proposed resolution:

Change in 6.8.1 [[basic.types.general](#)] paragraph 6 as follows:

... [**Note:** The type of a pointer ~~or reference~~ to array of unknown bound **permanently points to or refers to an incomplete type. An array of unknown bound, or of a type defined named by a typedef declaration to be an array of unknown bound, permanently refers to an incomplete type. In either case, the array type cannot be completed. -- end note**]

2644. Incorrect comment in example

Section: 7.5.5.3 [[expr.prim.lambda.capture](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 13-042](#)

The comment in the example in 7.5.5.3 [[expr.prim.lambda.capture](#)] paragraph 6 refers to "local variable", but should refer to *init-capture* instead.

Possible resolution:

Change in 7.5.5.3 [[expr.prim.lambda.capture](#)] paragraph 6 as follows:

```
auto z = [a = 42](int a) { return 1; };    // error: parameter and conceptual local variable have the same name
```

2645. Unused term "default argument promotions"

Section: 7.6.1.3 [[expr.call](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 15-044](#)

The phrase "default argument promotions" is apparently unused.

Possible resolution [**SUPERSEDED**]:

1. Change in 7.6.1.3 [[expr.call](#)] paragraph 12 as follows:

...If the argument has integral or enumeration type that is subject to the integral promotions (7.3.7 [[conv.prom](#)]), or a floating-point type that is subject to the floating-point promotion (7.3.8 [[conv.fpprom](#)]), the value of the argument is converted to the promoted type before the call. ~~These promotions are referred to as the default argument promotions.~~

2. Change in 17.13.2 [[cstdarg.syn](#)] as follows:

See also: ISO C ~~7.16.1.1~~ **7.16.1**

CWG 2022-11-10

The phrase is useful and needed to override C's deviating definition, as applicable to `va_arg`.

Proposed resolution (approved by CWG and LWG 2022-11-11):

Change in 17.13.2 [[cstdarg.syn](#)] as follows:

The contents of the header `<cstdarg>` are the same as the C standard library header `<stdarg.h>`, with the following changes: **In lieu of the default argument promotions specified in ISO C 6.5.2.2, the definition in 7.6.1.3 [[expr.call](#)] applies.** The restrictions that ISO C places on the second parameter to the `va_start` macro in header `<stdarg.h>` are different in this document. ...

2646. Defaulted special member functions

Section: 9.5.2 [[dcl.fct.def.default](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 20-052](#)

The exposition in terms of F1 and F2 as well as T1 and T2 is confusing.

Possible resolution:

Change in 9.5.2 [[dcl.fct.def.default](#)] paragraph 2 as follows:

An explicitly defaulted special member function F1 ~~with type T1~~ is allowed to differ from the corresponding special member function F2 ~~with type T2~~ that would have been implicitly declared, as follows:

- ~~T1~~ **F1** and ~~T2~~ **F2** may have differing *ref-qualifier*s;
- if F2 has an implicit object parameter of type “reference to C”, F1 may be an explicit object member function whose explicit object parameter is of type “reference to C”, in which case ~~T1~~ **the type of F1** would differ from ~~T2~~ **the type of F2** in that ~~T1~~ **the type of F1** has an additional parameter;
- ~~T1~~ **F1** and ~~T2~~ **F2** may have differing exception specifications; and
- if F2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F1 may be of type `C&`.

If ~~T1~~ **the type of F1** differs from ~~T2~~ **the type of F2** in a way other than as allowed by the preceding rules, then:

- if F1 is an assignment operator, and the return type of ~~T1~~ **F1** differs from the return type of ~~T2~~ **F2** or F1's non-object parameter type is not a reference, the program is ill-formed;
- otherwise, if F1 is explicitly defaulted on its first declaration, it is defined as deleted;
- otherwise, the program is ill-formed.

2647. Fix for "needed for constant evaluation"

Section: 7.7 [[expr.const](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 17-047](#)

The criteria for a variable to be needed for constant evaluation are inconsistent with those for it to be usable in constant expressions (/4).

Proposed resolution (approved by CWG 2022-11-08):

1. Change in 7.7 [[expr.const](#)] paragraph 3 as follows:

A variable is *potentially-constant* if it is constexpr or it has reference or **non-volatile** const-qualified integral or enumeration type.

2. Change in 7.7 [expr.const] paragraph 16.7 as follows:

A function or variable is needed for constant evaluation if it is:

- a constexpr function that is named by an expression (6.3) that is potentially constant evaluated, or
- a **potentially-constant** variable named by a potentially constant evaluated expression ~~that is either a constexpr variable or is of non-volatile const-qualified integral type or of reference type.~~

2648. Correspondence of surrogate call function and conversion function

Section: 12.4.4 [over.call] **Status:** NB **Submitter:** CA **Date:** 2022-11-03 [P2720R0 comment CA 063](#)

Post-Prague "editorial" change cplusplus/draft#3625 removed the normative text supporting the interpretation that surrogate call functions, when chosen, call the functions they were formed from.

Proposed resolution (approved by CWG 2022-11-08):

Change in 12.4.4 [over.call] paragraph 1 as follows:

If a surrogate call function ~~for a conversion function named operator conversion-type-id~~ is selected, **let *e* be the result of invoking the corresponding conversion operator function on the postfix-expression;** ~~is invoked~~ the expression is interpreted as

~~postfix-expression . operator conversion-type-id ()~~ ***e*** (expression-list_{opt})

Otherwise, ...

2649. Incorrect note about implicit conversion sequence

Section: 12.2.2.2.3 [over.call.object] **Status:** NB **Submitter:** CA **Date:** 2022-11-03 [P2720R0 comment CA 064](#)

Contrary to the note in the subject paragraph, overload resolution in selecting a surrogate call function can prefer a different conversion operator for the implicit conversion sequence because the conversion function from which the surrogate call function was derived is not the best viable function.

For example, noting that surrogate call functions are not generated from conversion function templates, the single surrogate call function derived from the (non-template) conversion function below is the sole candidate for the call; however, the specialization of the conversion function template is a better candidate for the implicit conversion sequence during overload resolution:

```
using ff = int (*)(int);
constexpr int ffimpl0(int x) {
    return x;
}
constexpr int ffimpl1(int x) {
    return x + 1;
}
struct A {
    template <typename T> constexpr operator T() const { return ffimpl0; }
    constexpr operator ff() const volatile { return ffimpl1; }
};
char x[A()(42.f)];
extern char x[43];
```

Proposed resolution (approved CWG 2022-11-08):

Change in 12.2.2.2.3 [over.call.object] paragraph 3 as follows:

[Note 1: When comparing the call against the function call operators, the implied object argument is compared against the object parameter of the function call operator. When comparing the call against a surrogate call function, the implied object

argument is compared against the first parameter of the surrogate call function. The conversion function from which the surrogate call function was derived will be used in the conversion sequence for that parameter since it converts the implied object argument to the appropriate function pointer or reference required by that first parameter. —end note]

2650. Incorrect example for ill-formed non-type template arguments

Section: 13.10.3.1 [[temp.deduct.general](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 28-068](#)

The example intends to illustrate that a class type cannot be the type of a non-type template parameter (although the example is still ill-formed because "T()" is interpreted as a function type).

Proposed resolution (approved by CWG 2022-11-08):

Change in 13.10.3.1 [[temp.deduct.general](#)] bullet 11.8 as follows:

- ...
 - Attempting to give an invalid type to a non-type template parameter. [Example 13:

```
template <class T, T> struct S {};
template <class T> int f(S<T, T()T{}> *); // #1
class X {
    int m;
};
int i0 = f<X>(0); // #1 uses a value of non-structural type X as a non-type template argument
```

-- end example]
 - ...
-

2651. Conversion function templates and "noexcept"

Section: 13.10.3.4 [[temp.deduct.conv](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 29-069](#)

The rule in 13.10.3.4 [[temp.deduct.conv](#)] bullet 5.2 seems to allow

```
template<class T, bool B>
using get=T(*)() noexcept(B);

struct A {
    template<class T>
    operator get<T, false>() const;
};

auto *p=A().operator get<int, true>();
```

Proposed resolution (approved by CWG 2022-11-09):

Change in 13.10.3.4 [[temp.deduct.conv](#)] paragraph 1 as follows:

... If the *conversion-function-id* is constructed during overload resolution ([over.match.funcs]), the following transformations rules in the remainder of this subclause apply.

Change in 13.10.3.4 [[temp.deduct.conv](#)] bullet 5.2 as follows:

However, certain attributes of A may be ignored:

- ...
- If the original A is a function pointer or pointer-to-member-function type with a potentially-throwing exception specification (14.5 [[except.spec](#)]), its `noexcept` the exception specification.
- ...

2652. Overbroad definition of `__STDCPP_BFLOAT16_T`

Section: 15.11 [[cpp.predefined](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 31-071](#)

The wording for the predefined macro `__STDCPP_BFLOAT16_T` added by P1467 can be interpreted more broadly than was intended.

Proposed resolution (approved by CWG 2022-11-08):

Change in 15.11 [[cpp.predefined](#)] paragraph 1 as follows:

`__STDCPP_BFLOAT16_T`

Defined as the integer literal 1 if and only if the implementation supports an extended floating-point type with the properties **of the typedef-name `std::bfloat16_t` as** described in 6.8.3 [[basic.extended.fp](#)].

2653. Can an explicit object parameter have a default argument?

Section: 9.3.4.6 [[dcl.fct](#)] **Status:** NB **Submitter:** GB **Date:** 2022-11-03 [P2720R0 comment GB 051](#)

The syntax and semantics appear to allow:

```
struct S { void f(this const S& = S{}); };
```

This is probably no more than an oddity, but perhaps it should be prevented.

Proposed resolution (approved by CWG 2022-11-08):

Change in 9.3.4.6 [[dcl.fct](#)] paragraph 3 as follows:

parameter-declaration:

```
attribute-specifier-seqopt thisopt decl-specifier-seq declarator  
attribute-specifier-seqopt thisopt decl-specifier-seq declarator = initializer-clause  
attribute-specifier-seqopt thisopt decl-specifier-seq abstract-declaratoropt  
attribute-specifier-seqopt thisopt decl-specifier-seq abstract-declaratoropt = initializer-clause
```

2654. Un-deprecation of compound volatile assignments

Section: 7.6.19 [[expr.ass](#)] **Status:** NB **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 16-045](#)

Many functions of volatile were deprecated in C++20. In C++23, volatile compound operations were de-deprecated by P2327. The rationale for this de-deprecation is lacking.

Deprecation is not removal. P2327 in C++23 is stopping the change that we began with C++20, mere moments ago if counting by adoption time. It primarily argued that it's an inconvenient change, and that some of the audience would just not cooperate with WG21's indicated direction, so WG21 should compromise the technical consistency of the Standard so they could continue to not cooperate.

The paper did not bring new information on the technical merits of the case, and net the result of applying it was a technically dissonant Standard specification --- a strictly worse specification than C++20. Bitwise compound operations are not special for any technical reason, they are only special because making them special shields some from deprecation diagnostics.

EWG 2022-11-07

Contrary to the direction desired in the NB comment, EWG resolved to un-deprecate all volatile compound assignments.

Proposed resolution (approved by CWG 2022-11-08):

Change in 7.6.19 [[expr.ass](#)] paragraph 6 as follows:

The behavior of an expression of the form $E1 \text{ op} = E2$ is equivalent to $E1 = E1 \text{ op} E2$ except that $E1$ is evaluated only once.

[Note: The object designated by $E1$ is accessed twice. -- end note] Such expressions are deprecated if $E1$ has volatile-qualified type and op is not one of the bitwise operators $|$, $\&$, $\wedge$; see D.5.

Change in D.5 [[depr.volatile.type](#)] paragraph 2 as follows:

```
brachiosaur += neck;           // deprecatedOK
brachiosaur = brachiosaur + neck; // OK
brachiosaur |= neck;           // OK, bitwise compound expression
```